

ScaleScape Lab

Case Back-End Pleno Procob

Scraping distribuido, RabbitMQ, Celery, Playwright, PostgreSQL, observabilidade, CI/CD e deploy em VPS.

Documento executivo e tecnico para avaliacao de aderencia a uma vaga de Desenvolvedor Back-End Pleno.

ScaleScape Lab - Case Back-End Pleno Procob

Case tecnico criado por iniciativa propria para demonstrar aderencia a uma vaga de Desenvolvedor Back-End Pleno com foco em scraping em escala, mensageria, performance, observabilidade, Linux, Docker e investigacao de bloqueios. Este material nao representa parceria oficial com a Procob.

Resumo Executivo

O ScaleScape Lab e um laboratorio completo de scraping distribuido. Ele mostra como eu desenho, implemento e opero um pipeline de coleta com API, fila, workers, browser automation, persistencia, retry, DLQ, circuit breaker, observabilidade e deploy automatizado.

Para RH, a leitura rapida e: o projeto foi feito para provar conhecimento pratico nos pontos centrais da vaga. Para engenharia, o repositorio permite inspecionar codigo, arquitetura, testes, Docker, CI/CD, metricas e o fluxo de dados de ponta a ponta.

Links De Demonstracao

Ambiente	UI	Dashboard	API Docs	Grafana	RabbitMQ
Dev	https://dev.scalescape.cledson.com.br	https://dev.scalescape.cledson.com.br/dashboard	https://api-dev.scalescape.cledson.com.br/docs	https://grafana-dev.scalescape.cledson.com.br	https://rabbit-dev.scalescape.cledson.com.br
Producao	https://scalescape.cledson.com.br	https://scalescape.cledson.com.br/dashboard	https://api.scalescape.cledson.com.br/docs	https://grafana.scalescape.cledson.com.br	https://rabbit.scalescape.cledson.com.br

Acessos administrativos nao sao versionados no repositorio. Grafana usa usuario admin; RabbitMQ Management usa scalescape_viewer. As senhas ficam nos secrets de cada ambiente.

Como Este Case Conversa Com A Vaga

Requisito da vaga	Evidencia no projeto
Scraping em larga escala	Pipeline com API, filas, workers, agendamento e benchmark controlado.
Pipelines distribuidos com mensageria	RabbitMQ, filas <code>scrape.jobs</code> , <code>scrape.retry</code> e <code>scrape.dead_letter</code> , com Celery workers.
Alto volume de requisicoes	Script <code>tools/load_demo.py</code> dispara jobs concorrentes e mede throughput, status, retries e percentis.
Latencia e throughput	Metricas Prometheus, Grafana, controle de timeout e backoff exponencial.
Workers paralelos	Worker stateless, filas separadas e execucao horizontal via Docker Compose.
Rate limit, CAPTCHA, anti-bot e fingerprint	Simulador anti-bot controlado, reCAPTCHA de laboratorio, browser profile, cookies, sessao e debug artifacts.
Proxies e controle de trafego	Proxy manager logico, cooldown por fonte/proxy e suporte a proxy proprio via Tailscale.
HTTP, headers, cookies e sessoes	Playwright manipula login, cookies <code>lab_auth/lab_clearance</code> , headers e contexto de browser.
Retry, timeout e falhas	Retry policy, DLQ, status por job, eventos e circuit breaker por fonte.
Linux, Docker e deploy	Docker Compose, GHCR, GitHub Actions, VPS, Nginx externo e migrations Alembic.

Requisito da vaga	Evidencia no projeto
Monitoramento e logs	Prometheus, Grafana, RabbitMQ Management, job events e artefatos de debug.

Arquitetura Geral

Diagrama Mermaid

flowchart LR

```

Candidate["Usuario / Avaliador"] --> UI["Target Site Next.js"]
Candidate --> Swagger["FastAPI Swagger"]
Candidate --> Dash["Dashboard"]

```

```

GH["GitHub Actions"] --> GHCR["GHCR Images"]
GHCR --> VPS["VPS Docker Compose"]

```

```

VPS --> API["FastAPI API"]
VPS --> Site["Next.js Target Site"]
VPS --> Worker["Celery Workers"]
VPS --> Beat["Celery Beat Scheduler"]
VPS --> MQ["RabbitMQ"]
VPS --> DB["PostgreSQL"]
VPS --> Prom["Prometheus"]
VPS --> Graf["Grafana"]

```

```

API --> DB
API --> MQ
Beat --> API
MQ --> Worker
Worker --> Playwright["Playwright Chromium"]
Playwright --> Site
Playwright --> External["Books / Globo / Betano"]
Worker --> DB
API --> Prom
Worker --> Prom
Prom --> Graf

```

Fluxo Completo De Scraping

Diagrama Mermaid

sequenceDiagram

```

participant User as Usuario/Dashboard
participant API as FastAPI
participant DB as PostgreSQL
participant MQ as RabbitMQ
participant W as Celery Worker
participant B as Playwright
participant T as Fonte/Target

```

```

User->>API: POST /jobs
API->>DB: cria job + job_event
API->>MQ: publica tarefa scrape.jobs
MQ->>W: entrega job
W->>DB: marca running + proxy_selected
W->>B: abre Chromium
B->>T: acessa URL inicial
T-->>B: login/challenge/dados
B->>T: envia credenciais/captcha quando aplicavel
B-->>W: dados extraidos
W->>DB: salva scraped_items idempotentes
W->>DB: atualiza status e items_found
User->>API: GET /jobs/{id} ou /items/page
API-->>User: resultado paginado

```

Falhas, Retry, DLQ E Circuit Breaker

Diagrama Mermaid

```
flowchart TD
    Job["Job em execucao"] --> Outcome{"Resultado"}
    Outcome --> Success["success + itens salvos"]
    Outcome --> Block["403 blocked"]
    Outcome --> Rate["429 rate_limited"]
    Outcome --> Timeout["timeout/erro tecnico"]
    Outcome --> Policy["blocked_by_policy"]

    Block --> Retry{"Tentativas restantes?"}
    Rate --> Retry
    Timeout --> Retry

    Retry -->|sim| Backoff["retry com backoff"]
    Backoff --> Queue["scrape.retry"]
    Queue --> Job

    Retry -->|nao| DLQ["scrape.dead_letter"]
    DLQ --> Circuit["source circuit_open"]
    Policy --> Stop["falha sem retry"]
```

O objetivo não é prometer coleta garantida em qualquer site. O objetivo é demonstrar a engenharia por trás de um sistema que reconhece bloqueios, registra diagnósticos, evita loops agressivos, aplica cooldown e mantém a operação observável.

Anti-Bot, CAPTCHA E Diagnostico De Bloqueios

A vaga cita contorno de bloqueios, CAPTCHA, anti-bot e fingerprint. Neste caso, esses temas aparecem de forma explícita e responsável:

- o target-site próprio exige login, cookies de sessão e CAPTCHA de laboratório;
- o worker detecta formulário de login, preenche credenciais e resolve o desafio

configurado para o ambiente controlado;

- o simulador anti-bot local gera cenários de atraso, challenge, 403, 429 e

mudança de layout;

- o browser profile configura sinais de sessão, headers, idioma, viewport e

contexto do Chromium;

- a fonte Betano possui diagnóstico de bloqueio, proxy próprio/Tailscale e

artefatos de debug para entender se houve 403, redirect, página vazia ou mudança de markup;

- qualquer uso externo deve ser autorizado, controlado e em baixo volume.

O projeto fala sobre anti-bot e bloqueios porque isso é parte do trabalho de engenharia pedido na vaga. Ele não documenta um passo a passo para violar controles de terceiros.

Persistencia E Evolucao De Banco

O banco usa PostgreSQL com SQLAlchemy e Alembic. Jobs, eventos, fontes, itens extraídos, desafios de CAPTCHA, eventos anti-bot e perfis de proxy ficam modelados no schema inicial.

Os itens são persistidos de forma idempotente por `job_id`, com `created_at` e `extracted_at`. Isso evita duplicidade por reprocessamento e deixa claro quando cada dado foi coletado.

Observabilidade

Diagrama Mermaid

```
flowchart LR
    API["FastAPI metrics"] --> Prom["Prometheus"]
    Worker["Worker metrics"] --> Prom
    MQ["RabbitMQ Management"] --> Operator["Avaliador"]
    Prom --> Graf["Grafana"]
    DB["job_events / statuses"] --> API
    API --> Dashboard["Dashboard Next.js"]
```

Durante uma demonstracao, e possivel abrir:

- dashboard para ver jobs e dados extraídos;
- Swagger para criar jobs e consultar status;
- RabbitMQ para ver filas, consumers e mensagens;
- Grafana para acompanhar metricas;
- logs/artefatos para diagnosticar falhas.

Benchmark De Carga

O script `tools/load_demo.py` dispara N jobs concorrentes contra a API e acompanha o status ate fim. Ele imprime:

- total enviado;
- sucessos, falhas e bloqueios;
- tempo total;
- jobs por minuto;
- itens coletados;
- duracao media, p50 e p95;
- retries/tentativas;
- resumo das filas RabbitMQ quando a Management API e informada.

Exemplo:

Trecho tecnico

```
python tools/load_demo.py --api-url http://localhost:8000 --jobs 30 --concurrency 10 --rabbitmq-url http://localhost:15672
```

Deploy E Operacao

O deploy usa GitHub Actions para:

- rodar testes Python e TypeScript;
- buildar API, worker e target-site;
- publicar imagens no GHCR;
- acessar a VPS por SSH;
- atualizar `.env.production`;
- rodar `alembic upgrade head`;
- subir Docker Compose;
- executar smoke de target-site, API, Grafana, RabbitMQ e scraping controlado.

Os servicos publicos ficam atras de Nginx/Certbot na VPS. Postgres, AMQP e Prometheus permanecem internos.

Fontes Demonstradas

Fonte	Objetivo
fake-target	Target controlado com login, CAPTCHA, anti-bot e paginacao.
books-to-scrape	Fonte publica segura para demonstrar parsing, detalhe e conversao de preco.
globo-home	Noticias publicas com categoria, resumo, link e imagem local.

Fonte	Objetivo
betano-football	Fonte mais difícil para demonstrar diagnóstico de bloqueio, browser e proxy.

Roteiro De Demonstracao Em 5 Minutos

1. Abrir <https://dev.scalescrape.cledson.com.br/dashboard>.
2. Mostrar jobs recentes e as abas de dados extraídos.
3. Clicar em uma ação de scraping manual.
4. Abrir Swagger e acompanhar GET /jobs/{id}.
5. Abrir RabbitMQ e mostrar filas/consumers.
6. Abrir Grafana e mostrar mettricas.
7. Explicar retry, DLQ, circuit breaker e idempotencia.
8. Rodar ou mostrar o benchmark controlado.

Onde Olhar No Repositorio

- API: `apps/api/app`
- Worker: `apps/worker/app`
- Target-site/dashboard: `apps/target_site/src`
- Infra: `docker-compose.yml`, `compose.deploy.yml`, `infra/`
- CI/CD: `.github/workflows`
- Benchmark: `tools/load_demo.py`
- Fluxo tecnico detalhado: [docs/fluxo-scraping.md](#)

Fechamento

Este case foi construido para mostrar autonomia tecnica, perfil investigativo e capacidade de operar uma arquitetura de scraping com volume, falhas reais, mensageria, observabilidade e deploy. A parte visual ajuda o RH a entender o valor rapidamente; o código e a infraestrutura permitem a avaliação tecnica em profundidade.